

Coding Activity: Hotel Billing System

Description

LuxeStay is a luxury hotel chain that wants to automate its billing system for different types of rooms. You are building a Hotel Billing System that allows users to enter room details, calculate the total bill based on the number of nights stayed, and check if the stay qualifies for a loyalty discount based on the number of years since joining their membership program.

Component Specification: Room Interface

Type (Interface)	Methods	Responsibilities
Room	<pre>public double calculateTotalBill(int nightsStayed, int joiningYear)</pre>	This method should calculate and return the total bill for a room based on the number of nights stayed. It should be an abstract method and give implementation inside the <code>HotelRoom</code> class.
Room	<pre>public default int calculateMembershipYears(int joiningYear)</pre>	This method should calculate and return the number of years since joining as a member (using the current year).

Component Specification: HotelRoom Class (Implements Room Interface)

Attributes

Type (Class)	Attributes	Responsibilities
HotelRoom	<pre>String roomType double ratePerNight String guestName</pre>	Appropriate setters, getters, and three-argument constructor are provided as part of code skeleton.

Component Specification: HotelRoom Class – Methods

Type (Class)	Methods	Responsibilities
HotelRoom	<pre>public double calculateTotalBill(int nightsStayed, int joiningYear)</pre>	• Calculates and returns the total bill for a guest based on nights stayed. • Formula: $\text{TotalBill} = \text{nightsStayed} \times \text{ratePerNight}$ • If membership is more than 3 years, apply 10% discount (guest pays 90%). • Final bill must be rounded to nearest whole number before returning.

UserInterface Class – Main Method Instructions

In the `UserInterface` class:

1. Prompt the user to enter details for both room types — **Deluxe Room** and **Suite Room** — following the format shown in the sample input and output.
 2. Initialize the values to `HotelRoom` class based on the room type (Deluxe or Suite).
 3. Based on the entered details, create `HotelRoom` objects and initialize:
 - `roomType`
 - `ratePerNight`
 - `guestName`
 4. Invoke the default method `calculateMembershipYears()` by passing the guest's joining year.
 5. Invoke the method `calculateTotalBill()` by passing:
 - number of nights stayed
 - joining year
 6. Calculate the final bill amount, applying discounts when applicable.
 7. Display a summary of each guest's room details, membership duration, and total bill in the specified output format.
-

Note

- `roomType` is case-sensitive.
- In the Sample Input / Output provided, the highlighted text in bold corresponds to the input given by the user, and the rest of the text represents the output.
- Ensure to follow the object-oriented specifications provided in the question description.

- Ensure to provide the names for classes, attributes, and methods as specified.
 - Adhere to the code template, if provided.
 - Please do not use `System.exit(0)` to terminate the program.
-

Sample Input and Output 1

Input

Enter Deluxe Room Details:

Guest Name: Suresh
Rate per Night: 8999
Nights Stayed: 3
Joining Year: 2019

Enter Suite Room Details:

Guest Name: Sunita
Rate per Night: 9999
Nights Stayed: 4
Joining Year: 2024

Output

Room Summary:

Deluxe Room: Suresh, 8999.0 per night, Membership: 6 years
Suite Room: Sunita, 9999.0 per night, Membership: 1 years

Total Bill:

For Suresh (Deluxe): 24297.0
For Sunita (Suite): 39996.0

Coding Activity: XamXpert

Description

XamXpert is an online examination platform that wants to automate its test evaluation process for different types of questions. You are building a Test Evaluation System that allows users to enter exam details, calculate the total score percentage based on correct and wrong answers, and determine the final result (Merit, Pass, or Fail) using a static method defined in an interface.

Component Specification: Exam Interface

Type	Methods	Responsibilities
Interface	<pre>public double calculateScore()</pre>	This is an abstract method to calculate and return the score percentage for a student based on the question type, number of correct and wrong answers. The implementation is provided inside the <code>OnlineTest</code> class.
Interface	<pre>public static String evaluateResult(double percentage)</pre>	This is a static method that evaluates and returns the final result (Merit, Pass, or Fail) based on the percentage score. • If percentage $\geq 85 \rightarrow$ "Merit" • If percentage ≥ 60 and $< 85 \rightarrow$ "Pass" • Otherwise $\rightarrow$ "Fail"

Component Specification: OnlineTest Class (Implements Exam Interface)

Attributes

Type (Class)	Attributes	Responsibilities
OnlineTest	<pre>String studentName int totalQuestions</pre>	Appropriate setters, getters, and five-argument constructor are provided as part of the code skeleton.

Type (Class)	Attributes	Responsibilities
	int correctAnswers int wrongAnswers String questionType	

Component Specification: OnlineTest Class – Methods

Component Name	Type (Class)	Methods	Responsibilities
Returning total score percentage	OnlineTest	<pre>public double calculateScore()</pre>	This method calculates and returns the percentage score of a student based on the question type and the number of correct and wrong answers. Rules: • For MCQ type → each correct answer carries 2 marks. • For Coding type → each correct answer carries 5 marks. • For every wrong answer → 10% of marks per question is deducted as penalty. Formula: <pre>totalScore = (correctAnswers × marksPerQuestion) - (wrongAnswers × (marksPerQuestion × 0.10))</pre> <pre>percentage = (totalScore / (totalQuestions × marksPerQuestion)) × 100</pre>

Example

If there are 10 coding questions, out of which 7 are correct and 3 are wrong:

```
totalScore = (7*5) - (3*(5*0.10)) = 35 - 1.5 = 33.5
```

```
percentage = (33.5 / (10*5)) * 100 = 67%
```

Hence, the student scored **67%** in the test.

UserInterface Class – Main Method Instructions

In the `UserInterface` class:

1. Prompt the user to enter the exam details such as:
 - Student name
 - Question type (MCQ or Coding)
 - Total number of questions
 - Number of correct answers
 - Number of wrong answers
2. Initialize the values into an `OnlineTest` object.
3. Invoke the `calculateScore()` method to compute the percentage score.
4. Invoke the static method `evaluateResult()` to determine whether the student has achieved:
 - Merit
 - Pass
 - Fail
5. Finally, display the result as shown in the sample output.

Note

- `questionType` is case-sensitive.
- In the output, the `totalScore` should be displayed with **exactly one decimal place**.
- In the Sample Input / Output provided, the highlighted text in bold corresponds to user input.
- Ensure to follow the object-oriented specifications given in the description.
- Ensure to use the same names for classes, attributes, and methods.
- Adhere to the code template, if provided.
- Please do not use `System.exit(0)` to terminate the program.

Sample Input and Output 1

Input

Enter Exam Details:

Student Name:
Sara Khan

Question Type (MCQ/Coding):

Coding

Total Questions:
20

Correct Answers:
18

Wrong Answers:
2

Output

Exam Summary:

Coding Test: Sara Khan, Total Score: 89.0, Result: Merit

Sample Input and Output 2

Input

Enter Exam Details:

Student Name:
Rio

Question Type (MCQ/Coding):
MCQ

Total Questions:
25

Correct Answers:
19

Wrong Answers:
6

Output

Exam Summary:

MCQ Test: Rio, Total Score: 73.6, Result: Pass